Real-Time Embedded Network Security Simulator

(COURSE PROJECT - BCSE305L)

By

24BYB0029	***Shikhar Sahay***
24BYB0111	***Ujjwal Pandit***
24BYB0186	***Ramsha Akbar***
24BYB0193	***Anwita Padhi***

Under the guidance of

Dr. Kalaiselvan N

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND INFORMATION SYSTEMS

TABLE OF CONTENTS

SERIAL NO.	SECTION TITLE	PAGE NO.
1	TITLE PAGE	1
2	ABSTRACT	3
3	INTRODUCTION: SCENARIO & RESEARCH GAP	4
4	INTRODUCTION: PROPOSED IDEA	5
5	METHODOLOGY	6
6	PROCESS FLOW DIAGRAM	8
7	CIRCUIT DIAGRAM	9
8	SIMULATION SETUP	11
9	SIMULATION/ALGORITHM	12
10	RESULTS AND DISCUSSION	14
11	CONCLUSION	18
12	REFERENCES	19

ABSTRACT

Modern embedded systems in safety-critical domains such as automotive control, industrial automation, and healthcare rely on reliable communication and strict real-time execution. With increasing interconnectivity, these systems are exposed to cyber threats that can disrupt timing behavior and compromise operational safety, making it essential to integrate security without violating real-time constraints.

This project presents the Real-Time Embedded Network Security Simulator (RT-ENSS), a SystemC-based framework that models a multi-node embedded system executing tasks under Rate Monotonic Scheduling (RMS) while communicating over a shared network. The simulator incorporates realistic communication patterns inspired by embedded protocols such as CAN and UART, and introduces cyber attacks, including spoofing, replay, and denial-of-service (DoS), to evaluate system behavior under adversarial conditions.

An intrusion detection system (IDS) is integrated into the execution pipeline to monitor network traffic and detect anomalies in real time, enabling adaptive responses such as safe-state transitions without violating task deadlines. The simulation output is visualized using a Python-based GUI dashboard, providing insights into scheduler activity, network behavior, and system response during attacks.

The results demonstrate that embedded systems can effectively detect and respond to cyber threats while maintaining real-time guarantees, highlighting the feasibility of integrating security mechanisms within time-critical environments.

INTRODUCTION

SCENARIO

Modern embedded systems are widely used in safety-critical domains such as automotive systems, industrial automation, and healthcare devices [1], [11]. These systems operate under strict real-time constraints, where timely task execution and reliable communication are essential [2], [14]. Communication between components typically occurs over protocols such as CAN, enabling coordination across multiple nodes [3], [4].

With increasing connectivity, embedded systems are becoming more vulnerable to cyber threats. Attacks such as spoofing, replay, and denial-of-service (DoS) can disrupt communication and interfere with time-critical tasks, potentially leading to system instability [5]–[7], [15]. Ensuring security is challenging because traditional mechanisms often introduce delays that violate real-time constraints [12]. Therefore, security must be integrated alongside deterministic scheduling without compromising system performance.

RESEARCH GAP

Most existing embedded system security solutions are hardware-intensive or offline, making them costly and difficult to prototype [10], [11]. Many approaches also fail to consider real-time scheduling constraints, which are critical in safety-critical applications [2]. While cryptographic techniques and hardware security modules provide protection, they are not always suitable for flexible simulation environments [12].

Additionally, offline methods cannot capture the interaction between scheduling, communication, and cyber attacks in real time. Existing intrusion detection systems often

target general-purpose networks and do not account for the timing and resource constraints of embedded systems [8], [9], leading to delayed or ineffective responses.

There is a lack of a unified simulation framework that integrates real-time scheduling, embedded communication, attack modeling, and intrusion detection. Such a framework is necessary to evaluate system behavior under adversarial conditions and study the trade-offs between security and real-time performance [13].

PROPOSED IDEA

To address these challenges, this project proposes the Real-Time Embedded Network Security Simulator (RT-ENSS), a SystemC-based framework for analyzing secure real-time embedded systems [3]. The system models multiple nodes executing tasks under Rate Monotonic Scheduling (RMS) while communicating over a shared network [1], [2].

The framework simulates communication and introduces cyber attacks such as spoofing, replay, and DoS at predefined times [5]–[7]. An intrusion detection system (IDS) monitors network activity and detects anomalies in real time, enabling timely response without violating task deadlines [8], [9].

A Python-based GUI dashboard visualizes node activity, scheduler execution, network traffic, and attack detection, providing an intuitive platform for analysis.

The system demonstrates that embedded environments can maintain real-time guarantees while effectively detecting and responding to cyber threats.

METHODOLOGY

RT-ENSS is designed as a unified simulation framework that integrates real-time scheduling, network communication, cyber attack modeling, and intrusion detection within a single execution environment. The system simulates a multi-node embedded architecture where all components operate under synchronized SystemC time, enabling accurate evaluation of system behavior under both normal and adversarial conditions.

At the core of the system is the Rate Monotonic Scheduling (RMS) algorithm, which assigns priorities to tasks based on their execution periods. Tasks with shorter periods are given higher priority, ensuring deterministic and predictable execution. The scheduler operates at each simulation tick, selecting and executing tasks while maintaining strict timing constraints and ensuring that high-priority tasks are serviced without delay.

Each node in the system represents an independent embedded unit executing periodic tasks. These nodes communicate over a shared network that simulates CAN-like behavior, where messages are transmitted with constraints such as delay, priority, and contention. This communication model reflects realistic embedded network conditions, including limited bandwidth and potential message collisions.

To evaluate system resilience, an attack injection module introduces controlled cyber threats during runtime. These include spoofing, replay, and denial-of-service (DoS) attacks, which are triggered at predefined timestamps. These attacks intentionally alter message patterns and network behavior, enabling systematic evaluation of how the system reacts to different types of adversarial conditions.

An intrusion detection system (IDS) is integrated into the execution loop to continuously monitor network traffic and task behavior. The IDS detects anomalies based on factors such as message validity, timing inconsistencies, and abnormal traffic patterns. Upon detecting suspicious activity, the system triggers appropriate response mechanisms, such

as safe-state transitions, while ensuring that task execution continues without violating real-time constraints.

All components operate in a closed-loop manner: the scheduler triggers task execution, nodes generate and exchange messages, the network manages communication, the attack module introduces disruptions, and the IDS monitors system behavior. This continuous interaction enables real-time observation of system dynamics and response mechanisms.

The resulting system activity is logged and processed by a Python-based GUI dashboard, which visualizes scheduler activity, node states, network traffic, and detection events. This provides a clear and interactive representation of system behavior, making it easier to analyze the impact of attacks and the effectiveness of detection mechanisms.

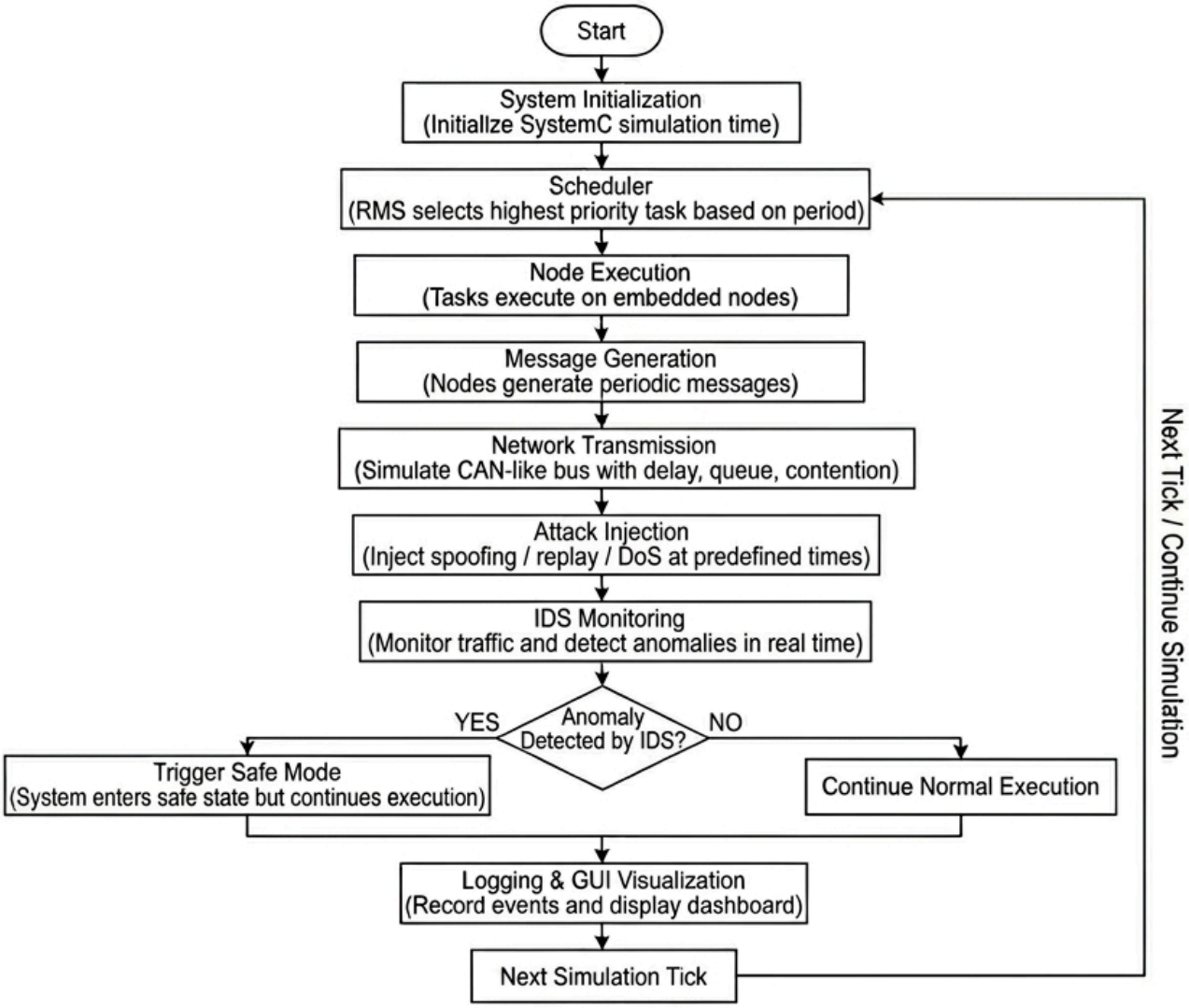
Overall, this integrated methodology enables comprehensive analysis of the interaction between scheduling, communication, and security, demonstrating that embedded systems can maintain real-time guarantees while effectively detecting and responding to cyber attacks.

PROCESS FLOW DIAGRAM

RT-ENSS begins with SystemC-based initialization, followed by RMS scheduling to execute tasks across multiple nodes. Messages are transmitted over a shared network with delay and contention.

An attack module injects spoofing, replay, and DoS events, while an IDS monitors traffic and detects anomalies in real time. Upon detection, a safe-state response is triggered without interrupting execution.

All activity is logged and visualized via a GUI dashboard, and the process repeats each simulation tick continuously.



CIRCUIT DIAGRAM

Since RT-ENSS is a software-based system, it does not involve physical circuitry. Instead, a logical system architecture is used to represent the interaction between different components of the embedded environment.

The system consists of multiple virtual nodes connected through a shared communication network, along with a scheduler, intrusion detection system (IDS), attack injection module, and a visualization interface. Each node executes periodic tasks and generates messages, which are transmitted over a network that simulates real embedded communication constraints such as delay, contention, and limited capacity.

The scheduler controls task execution using Rate Monotonic Scheduling (RMS), ensuring that timing constraints are maintained across all nodes. The attack injection module introduces controlled anomalies such as spoofing, replay, and denial-of-service (DoS) attacks to evaluate system behavior under adverse conditions.

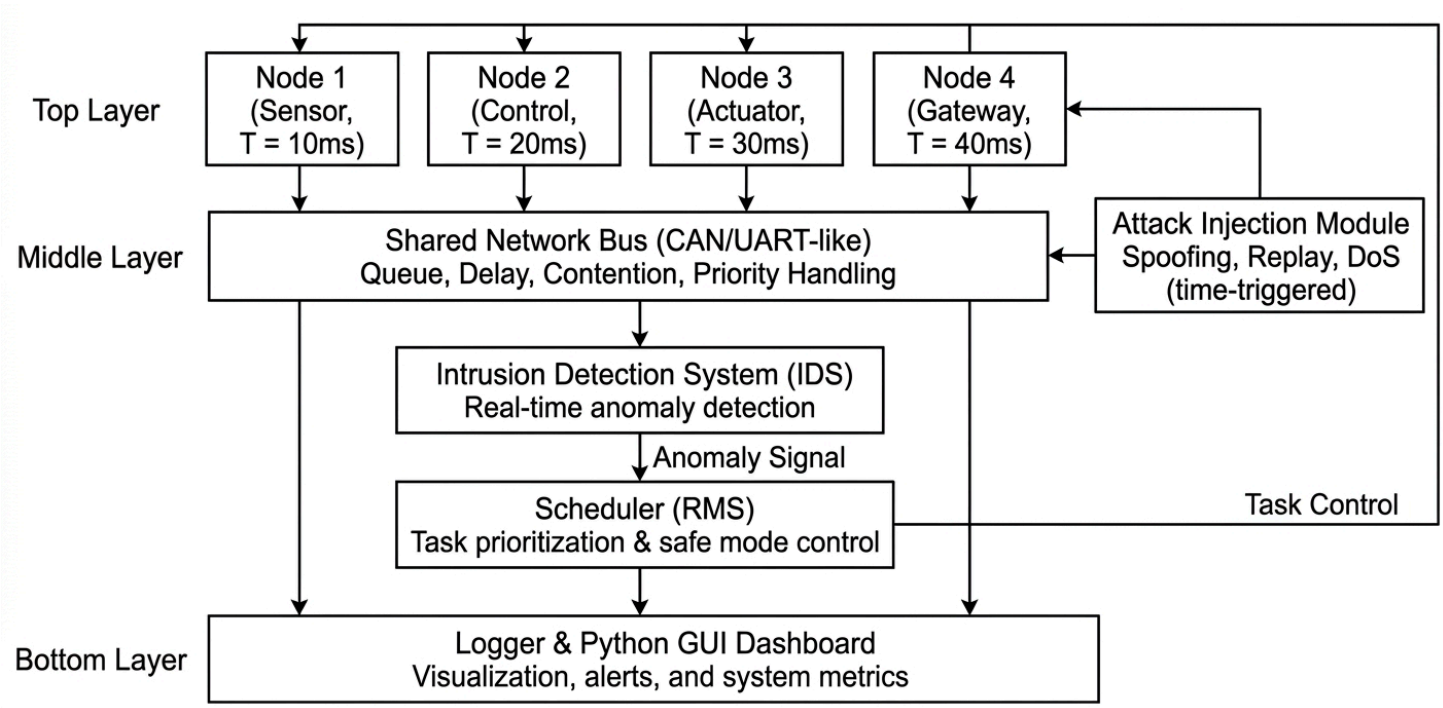
The IDS continuously monitors network traffic and task execution to detect anomalies in real time, while a logging and GUI module provides visualization of system activity and detection events. This logical representation captures the structure and interaction of components, enabling analysis of system performance without requiring physical hardware.

RT-ENSS MODULE DESCRIPTION

Module	Role	Interaction / Function
Node Module	Executes tasks with timers and interrupts.	Sends and receives network messages; responds to scheduler triggers
Scheduler	Implements RMS	Selects and prioritizes tasks each tick to

Module	real-time scheduling.	meet timing constraints.
Network Module	Simulates CAN/UART-like bus communication.	Transfers messages with delay, priority, and contention handling.
Attack Injection Module	Generates controlled cyber attacks.	Injects spoofed, replayed, or DoS traffic at defined timestamps.
IDS Module	Monitors network traffic and task behavior.	Detects anomalies based on timing, message patterns, and node activity.
Logger & Visualization (Python GUI)	Records and displays system activity.	Logs execution timelines, traffic, alerts, and feeds data to the GUI dashboard.

SYSTEM ARCHITECTURE DIAGRAM



SIMULATION SETUP

The RT-ENSS simulation is developed using SystemC 3.0.2 and compiled with GCC 15.2 via the MSYS2 environment. The entire system runs on a Windows platform without requiring any physical hardware, making it a fully software-based simulation framework.

The implementation consists of multiple modules organized as six header files along with a top-level main.cpp, which integrates all components of the simulation. In addition, a Python-based GUI module built using Tkinter is used for real-time visualization of system behavior.

To execute the simulation, the following tools and dependencies are required:

- SystemC 3.0.2 built from source
- GCC 15.2 with C++17 support via MSYS2
- Python (Tkinter) for GUI-based visualization
- GTKWave for waveform visualization
- PowerShell to execute the included run.ps1 build script

The project consists of six header files and a top-level main.cpp for the SystemC simulation, along with a Python-based GUI module for visualization. The run.ps1 script compiles and executes the simulation, generating a CLI event log and a VCD trace for GTKWave.

Four nodes run as concurrent threads, exchanging messages over a shared network bus. Each message includes a sender ID, payload, and timestamp for real-time IDS monitoring. The simulation runs for 300 ms with attack events at predefined timestamps, and the logs are processed by a Python GUI dashboard for visualization and analysis.

SIMULATION & ALGORITHM

RT-ENSS is designed around a unified real-time execution model where computation, communication, and security operate under the same timing constraints. The system integrates multiple tightly coupled modules, including the scheduler, nodes, network, attack injection module, and intrusion detection system (IDS), all operating within a synchronized 300 ms SystemC timeline.

At the core of the system is the Rate Monotonic Scheduling (RMS) algorithm, which assigns priorities based on task periods. Tasks with shorter periods are given higher priority, allowing deterministic execution and ensuring that critical tasks meet their deadlines. The scheduler operates at each simulation tick, selecting and executing tasks while maintaining strict timing constraints.

Each node executes periodic tasks and generates messages at fixed intervals. These messages contain sender ID, payload, sequence number, and timestamp, enabling accurate tracking of communication patterns. The nodes communicate through a shared network bus that simulates CAN/UART-like behavior, including delay, contention, and priority-based transmission.

To evaluate system resilience, an attack injection module introduces controlled cyber threats during runtime. These include spoofing, replay, and denial-of-service (DoS) attacks, which are triggered at predefined timestamps and alter normal communication patterns.

The intrusion detection system (IDS) operates within the same execution loop, continuously monitoring network traffic and task behavior in real time. It evaluates message validity, timing consistency, and traffic patterns to identify anomalies. Based on these checks, the IDS generates an anomaly score and signals the system when suspicious activity is detected.

Instead of treating security as an external add-on, RT-ENSS embeds detection directly into the scheduling and communication loop. This ensures that intrusion detection operates within bounded latency while preserving real-time task execution.

When anomalies are detected, the system activates safe modes that modify execution behavior without interrupting critical tasks. Non-critical tasks may be suspended, while essential tasks continue execution under RMS scheduling, ensuring system stability even under attack conditions.

RUNTIME EXECUTION AND DETECTION LOGIC

The simulation progresses over a 300 ms virtual timeline, where all modules execute concurrently under synchronized SystemC time. The scheduler selects tasks using RMS, and nodes generate periodic messages that are transmitted over a shared network, while the IDS continuously monitors system activity in real time.

At 60 ms, a spoofing attack is introduced using an invalid sender ID. At 100 ms, a replay attack is simulated using previously transmitted messages, and at 140 ms, a denial-of-service attack is triggered through network flooding.

The IDS evaluates these events in real time by analyzing message validity and traffic behavior. Upon detecting anomalies, the system activates safe mode, modifying execution while ensuring that critical tasks continue without deadline violations.

All system activity, including task execution, attack events, and detection responses, is logged and visualized through the Python GUI dashboard, enabling analysis of system behavior under both normal and adversarial conditions.

RESULTS AND DISCUSSION

The RT-ENSS simulation was executed over a 300 ms virtual timeline, with system activity visualized through a Python-based GUI dashboard. The results show that the system maintains real-time performance while detecting and responding to cyber attacks.

NORMAL OPERATION (0-60 MS)

The RT-ENSS simulation was executed over a 300 ms virtual timeline, with system activity visualized through a Python-based GUI dashboard. The results show that the system maintains real-time performance while detecting and responding to cyber attacks.

SPOOFING ATTACK (60 MS)

At 60 ms, a spoofed message with an invalid sender ID was injected. The IDS detected the anomaly and triggered safe mode shortly after detection. Task execution continued without interruption, maintaining real-time constraints.

REPLAY ATTACK (100 MS)

At 100 ms, previously transmitted messages were replayed, causing inconsistencies in timing and sequence patterns. The IDS identified these anomalies and reinforced safe mode activation, effectively distinguishing malicious data from normal traffic.

DENIAL-OF-SERVICE ATTACK (140 MS)

At 140 ms, a DoS attack flooded the network with excessive messages, increasing the bus load. The IDS detected abnormal traffic and triggered protective mechanisms, while the scheduler ensured that critical tasks continued without deadline violations.

SYSTEM STABILITY AND PERFORMANCE

Across all phases, the system remained stable. Critical tasks were executed without missing deadlines, and the IDS operated with minimal latency, demonstrating compatibility between security mechanisms and real-time requirements.

GUI-BASED ANALYSIS

The GUI dashboard provided real-time visualization of system behavior, including safe mode transitions, scheduler execution, and network activity. These observations confirm effective detection and response under both normal and attack conditions.

SIMULATION OUTPUT

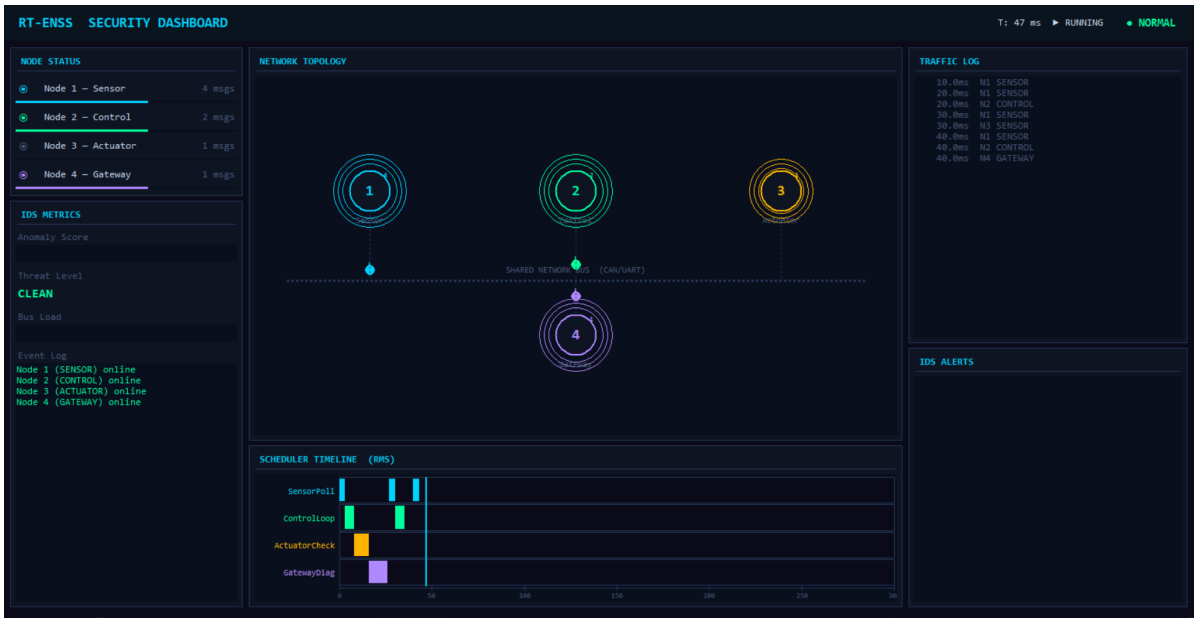


Fig 1: Normal system operation (~50 ms) under RMS scheduling

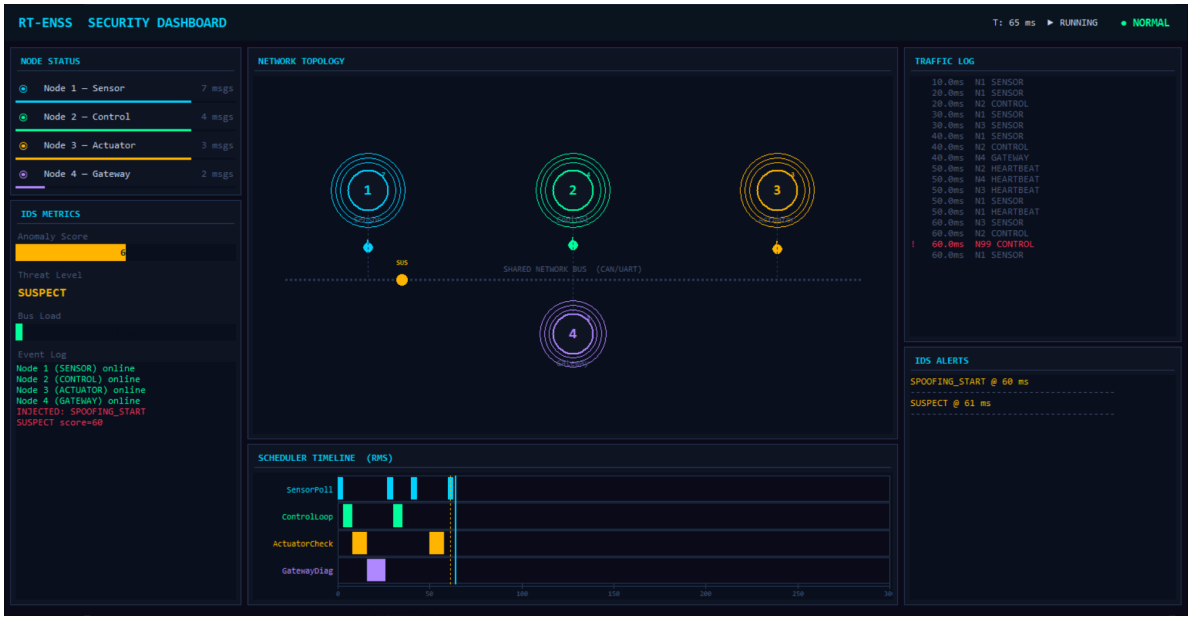


Fig 2: Spoofing attack detection and safe mode activation (~60 ms)

Fig 3: Replay attack detection with anomalous message patterns (~110 ms)

Fig 4: DoS attack causing network congestion and system response (~150 ms)

Fig 5: Final system state at 300 ms showing stable execution and recovery

CONCLUSION

The RT-ENSS framework demonstrates that real-time embedded systems can effectively detect and respond to cyber attacks without violating strict timing constraints. The simulation validates that the Rate Monotonic Scheduling (RMS) algorithm consistently executes tasks within deadlines, even during active attack conditions and safe mode operation.

The intrusion detection system (IDS) successfully identifies spoofing, replay, and denial-of-service (DoS) attacks in real time, triggering timely responses while maintaining uninterrupted execution of critical tasks. This confirms that security mechanisms can be integrated within the scheduling and communication framework without compromising real-time performance.

The project highlights the feasibility of implementing lightweight, rule-based intrusion detection in resource-constrained embedded environments. Additionally, the use of SystemC provides an effective platform for modeling, testing, and validating such systems before deployment.

Future work may include incorporating advanced detection techniques such as adaptive or learning-based IDS models, scaling the system to larger network configurations, and extending the framework to hardware-based implementations.

REFERENCES

[1] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” Journal of the ACM, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[2] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. New York, NY, USA: Springer, 2011.

[3] IEEE Standard for Standard SystemC Language Reference Manual, IEEE Std 1666-2011, Jan. 2012.

[4] Robert Bosch GmbH, CAN Specification Version 2.0, Sep. 1991.

[5] T. Hoppe, S. Kiltz, and J. Dittmann, “Security threats to automotive CAN networks—Practical examples and selected short-term countermeasures,” in Computer Safety, Reliability, and Security (SAFECOMP 2008), LNCS 5219, pp. 235–248, 2008.

[6] K. Koscher et al., “Experimental security analysis of a modern automobile,” in 2010 IEEE Symposium on Security and Privacy, pp. 447–462, 2010.

[7] S. Checkoway et al., “Comprehensive experimental analyses of automotive attack surfaces,” in Proceedings of the 20th USENIX Security Symposium, 2011.

[8] U. E. Larson, D. K. Nilsson, and E. Jonsson, “An approach to specification-based attack detection for in-vehicle networks,” in 2008 IEEE Intelligent Vehicles Symposium, pp. 220–225, 2008.

[9] M. Müter, A. Groll, and F. C. Freiling, “A structured approach to anomaly detection for in-vehicle networks,” in 2010 Sixth International Conference on Information Assurance and Security, pp. 92–98, 2010.

[10] I. Studnia, E. Alata, V. Nicomette, M. Kaâniche, and Y. Laarouchi, “A language-based intrusion detection approach for automotive embedded networks,” International Journal of Embedded Systems, vol. 10, no. 1, pp. 1–12, 2018.

[11] J. Rushby, “Critical system properties: Survey and taxonomy,” Reliability Engineering & System Safety, vol. 43, no. 2, pp. 189–219, 1994.

[12] P. Koopman, “Embedded system security,” IEEE Computer, vol. 37, no. 7, pp. 95–97, 2004.

[13] E. A. Lee, “Cyber physical systems: Design challenges,” in 2008 11th IEEE International Symposium on Object-Oriented Real-Time Distributed Computing (ISORC), pp. 363–369, 2008.

[14] A. Burns and A. J. Wellings, Real-Time Systems and Programming Languages, 4th ed. Addison-Wesley, 2009.

[15] S. Woo, H. J. Jo, and D. H. Lee, “A practical wireless attack on the connected car and security protocol for in-vehicle CAN,” IEEE Transactions on Intelligent Transportation Systems, vol. 16, no. 2, pp. 993–1006, 2015.